Journal de construction d'un socle cloud sécurisé

Azure Enterprise Security Lab

Comment on sécurise une migration cloud
quand on n'a ni budget, ni équipe, ni droit à l'erreur

Sunnoogo Caleb Tapsoba

Novembre 2026

Document narratif. La documentation technique du projet est tenue séparément.

Table des matières

Pourquoi ce document existe **2**

1 Le point de départ **3**

1.1 Une conviction née d'un échec . . . 3

1.2 Le parti pris qui en découle . . . 3

1.3 L'entreprise simulée . . . 3

2 Ce qu'on protège, avant de savoir comment **5**

2.1 Traduire une peur en exigences . . . 5

2.2 Une correction en cours de route . . . 5

3 Les premiers garde-fous **7**

3.1 Poser les protections avant de construire . . . 7

3.2 Un budget n'est pas un outil de comptabilité . . . 7

3.3 On n'attribue jamais un droit à une personne . . . 7

4 La mémoire de l'infrastructure **9**

4.1 Pourquoi cesser de travailler à la main . . . 9

4.2 Le carnet, et ce qu'il contient . . . 9

4.3 L'œuf et la poule . . . 10

4.4 Ce qui se joue dans un paramètre . . . 10

4.5 Administrer n'est pas lire . . . 10

4.6 Ne pas publier la carte . . . 11

4.7 Une ligne qui vaut une signature . . . 11

Journal et corrections **12**

Pourquoi ce document existe

Il existe déjà, quelque part dans ce dépôt, une documentation technique : des fichiers Markdown, des tableaux, des commandes, des rapports de bloc. Elle répond à la question *comment*. Elle est faite pour être consultée, pas pour être lue.

Ce document-ci répond à une autre question, et il se lit d'un bout à l'autre : *pourquoi*. Pourquoi ce projet a été entrepris, dans quel ordre les décisions ont été prises, ce qui a été écarté et à quel prix, et ce qu'on a compris en route qu'on ne savait pas au départ.

Il y a une raison précise à cette séparation. Une documentation technique montre un résultat. Elle ne montre jamais le raisonnement, parce que le raisonnement a été effacé au moment de la mise au propre. Or c'est le raisonnement qui a de la valeur : le résultat se copie, le raisonnement se défend.

> *Un projet d'école cherche la couverture : déployer le plus de services possible, cocher le plus de cases. Un projet d'ingénieur cherche la défendabilité : pouvoir justifier chaque ligne devant quelqu'un dont c'est le métier de la contester.*

Trois choix traversent tout ce travail, et il vaut mieux les énoncer au début.

On construit sous contrainte, et on le dit. Un compte d'essai, un seul abonnement, aucune licence avancée, des environnements détruits chaque soir pour ne rien payer la nuit. Ces limites ne sont pas des excuses reléguées en note de bas de page : elles sont le sujet. Chaque fois qu'un composant est écarté — pare-feu managé, accès administrateur sécurisé, haute disponibilité — ce document indique ce qu'on perd, et ce qu'on ferait à la place avec les moyens d'une PME, puis d'un grand compte. Travailler sans contrainte n'apprend rien, parce que personne ne travaille sans contrainte.

On arbitre, on ne cumule pas. Empiler des contrôles de sécurité est facile et donne l'illusion du sérieux. Le travail réel consiste à choisir entre des exigences qui se contredisent : conserver des journaux assez longtemps pour enquêter, mais pas au point de constituer soi-même une collecte abusive ; ouvrir assez de droits pour que les équipes avancent, pas assez pour qu'elles se nuisent. Ces arbitrages sont écrits, avec ce qu'ils coûtent.

On raconte les attaques plutôt que de les lister. Un tableau de menaces se lit et s'oublie. Le chapitre qui leur est consacré suit deux attaques de bout en bout, jusqu'à la facture et jusqu'aux obligations légales qu'elles déclenchent. C'est la seule façon de faire comprendre à une direction pourquoi une case cochée dans un portail vaut un budget.

CHAPITRE 1

Le point de départ

1.1 Une conviction née d'un échec

Je suis intervenu dans une entreprise qui venait de subir une attaque. Aujourd'hui encore, personne ne sait ce qui s'y est réellement passé. Pas parce que l'attaquant était remarquable. Parce qu'il n'y avait pas assez de journaux.

Cette phrase mérite qu'on s'y arrête, parce qu'elle décrit une situation bien pire qu'elle n'en a l'air. Une entreprise dans ce cas ne peut pas établir ce qui a été consulté. Elle ne peut donc pas restreindre sa communication aux clients réellement touchés : elle doit supposer le pire et prévenir tout le monde. Elle ne peut pas démontrer à un régulateur qu'elle maîtrise son système d'information, puisqu'elle vient d'administrer la preuve du contraire. Et elle ne peut pas clore l'incident, puisqu'elle ignore s'il est terminé.

Deux entreprises subissent la même attaque. La première a ses journaux : elle identifie quarante clients concernés, documente le périmètre, répond aux questions. La seconde ne les a pas : elle prévient deux cents clients et déclare qu'elle ne sait pas. Le préjudice technique est identique. Le préjudice réel n'a aucun rapport.

1.2 Le parti pris qui en découle

On ne se contente pas d'empêcher l'intrusion. On se donne les moyens de la reconstituer.

Empêcher l'accès est la partie du métier que tout le monde traite. Savoir, après coup, qui a fait quoi, quand, depuis où, est celle que tout le monde néglige — jusqu'au jour où elle manque.

De là découlent deux décisions, et la seconde est délibérément à contre-courant.

La **traçabilité** est traitée au même rang que la confidentialité : journalisation centralisée, supervision, et un bloc entier du projet consacré au traitement d'un incident réel, de la détection jusqu'au rapport.

La **disponibilité**, elle, est écartée du périmètre. C'est un choix de coût, et il est assumé : la disponibilité est déjà le réflexe de tout ingénieur d'exploitation. Un directeur s'inquiète d'un service interrompu le jour même ; personne n'est payé pour l'oublier. La traçabilité, elle, n'a pour défenseur que l'ingénieur sécurité — et elle disparaît dès qu'il se tait.

1.3 L'entreprise simulée

Le projet se déroule chez une PME française de conseil B2B : deux cents collaborateurs, trois sites, du télétravail généralisé. Elle migre vers Azure son portail client, qui tourne aujourd'hui sur un serveur

auto-hébergé sans sauvegarde testée ni supervision.

Ce portail héberge des audits, des rapports stratégiques, des analyses financières appartenant à ses clients. L'entreprise ne vend pas des documents : elle vend de la confiance. C'est une distinction qui paraît rhétorique jusqu'au jour d'une fuite, où elle devient la seule qui compte.

Le contexte est volontairement sobre. Pas de secteur réglementé exotique, pas de contrainte spectaculaire : le RGPD, et des engagements contractuels de confidentialité. Suffisant pour que chaque décision technique ait une conséquence réelle, assez neutre pour que l'attention reste sur l'ingénierie.

CHAPITRE 2

Ce qu'on protège, avant de savoir comment

2.1 Traduire une peur en exigences

La première question posée au dirigeant n'était pas technique : qu'est-ce qui vous empêche de dormir quand on vous annonce une migration cloud ? La réponse est venue sans hésitation, et sans un mot de jargon. Se faire pirater. Que les données des clients se retrouvent en de mauvaises mains. Que l'activité s'arrête au mauvais moment.

Le métier d'ingénieur sécurité commence exactement là : traduire cela en exigences. « Les données en de mauvaises mains » devient la **confidentialité**. « L'activité interrompue » devient la **disponibilité**. Deux propriétés sur quatre, énoncées spontanément.

Les deux manquantes sont toujours les mêmes, et leur absence est instructive. Personne ne pense spontanément à l'**intégrité** — qu'on modifie ses données sans qu'il le voie. Et personne, jamais, ne cite la **traçabilité** — pouvoir prouver ce qui s'est passé. C'est pourtant celle qui fait la différence le jour de l'incident, et c'est une obligation légale : l'article 33 du RGPD impose de notifier une violation de données personnelles dans les soixante-douze heures. Sans journaux centralisés, ce délai est intenable, et l'obligation devient une faute.

> *« Se faire pirater » n'est pas une menace : c'est une angoisse. La découper — qui, par où, pour obtenir quoi — c'est le modèle de menaces.*

2.2 Une correction en cours de route

Un raccourci s'était glissé dans le premier jet du cadrage : le principe du refus par défaut y était rattaché à l'article 5 du RGPD. C'est faux, et la correction vaut mieux que le raccourci.

L'article 5 énonce des principes relatifs aux *données* : minimisation de ce qu'on collecte, limitation de la durée de conservation, intégrité et confidentialité. Le texte qui correspond réellement au refus par défaut est l'**article 25**, « protection des données dès la conception et par défaut », dont le second alinéa dit en substance que, par défaut, les données ne doivent pas être rendues accessibles à un nombre indéterminé de personnes. C'est la formulation juridique exacte du principe technique.

Cette précision en ouvre une autre, plus intéressante. Ces principes n'ont pas été inventés par le droit. En 1975, Jerome Saltzer et Michael Schroeder publiaient huit principes de conception des systèmes sécurisés, dont le moindre privilège et les *valeurs par défaut sûres* — « fonder les décisions d'accès sur la permission plutôt que sur l'exclusion ». Le contrôle d'accès par rôles est formalisé en 1992. Azure les implémente autour de 2015. Le RGPD est adopté en 2016.

L'ingénierie avait donc quarante ans d'avance sur le législateur. Ce que le règlement a changé n'est pas technique : avant lui, le moindre privilège était une bonne pratique que l'on recommandait et que la direction arbitrait, souvent défavorablement. Depuis, c'est une obligation opposable, assortie d'une

sanction publique. Le RGPD n'a pas donné de nouveaux outils aux ingénieurs sécurité. Il leur a donné un budget.

CHAPITRE 3

Les premiers garde-fous

3.1 Poser les protections avant de construire

Rien n'était encore déployé. C'est précisément le moment de poser les garde-fous, parce qu'un garde-fou posé après l'accident ne sert plus à rien.

Trois questions, auxquelles toute plateforme devrait savoir répondre avant d'accueillir sa première ressource. Et si cela coûte trop cher ? Qui a le droit de faire quoi, et où ? Comment s'y retrouvera-t-on dans six mois ? C'est le travail le moins spectaculaire du projet, et celui dont l'absence se paie le plus cher.

3.2 Un budget n'est pas un outil de comptabilité

L'intuition commune range l'alerte budgétaire du côté de la gestion. C'est une erreur de lecture, et elle se démontre.

Un attaquant qui compromet des identifiants cloud ne vole pas toujours des données. Souvent, il déploie des machines de calcul pour miner de la cryptomonnaie aux frais de la victime, en choisissant des régions que l'entreprise n'utilise pas. Rien ne se déclenche : du point de vue de la plateforme, une identité autorisée crée des ressources qu'elle a le droit de créer. Pendant cinq semaines, le seul signal est la courbe de dépense.

> *L'alerte budgétaire fait partie du dispositif de détection. Le minage se voit sur la prévision de facture avant de se voir dans les journaux.*

D'où le détail qui compte dans la configuration retenue : un seuil sur la dépense *prévue*, et pas seulement sur la dépense *constatée*. Le premier prévient plusieurs jours à l'avance ; le second se contente de constater.

3.3 On n'attribue jamais un droit à une personne

Un rôle donné à un individu devient invisible. Il survit à son changement d'équipe, à son départ, et n'apparaît dans aucune procédure de sortie. Six mois plus tard, plus personne ne sait pourquoi cette personne a ce droit sur cette ressource — et plus personne n'ose le lui retirer.

Un rôle donné à un *groupe* se révoque en retirant la personne du groupe. Une seule action, au même endroit que toutes les autres, tracée.

Trois groupes ont donc été créés, qui correspondent à trois métiers réels : ceux qui construisent la plateforme, ceux qui déploient l'application, et ceux qui observent. Ce dernier mérite qu'on s'y arrête. L'analyste sécurité doit pouvoir consulter n'importe quelle ressource pour enquêter ; s'il peut aussi les

modifier, il peut effacer ses propres traces. On sépare celui qui observe de celui qui agit — exactement comme on sépare, en comptabilité, celui qui paie de celui qui contrôle.

Les groupes sont restés vides. C'est délibéré : on construit la structure, les occupants viendront plus tard. Et le dernier bloc du projet mettra précisément cette structure à l'épreuve, avec un incident d'escalade de privilèges — quelqu'un qui s'ajoute lui-même à un groupe trop puissant, en dehors de toute procédure. Ce sera à la traçabilité de le démasquer.

CHAPITRE 4

La mémoire de l'infrastructure

4.1 Pourquoi cesser de travailler à la main

Tout ce qui précède a été construit en tapant des commandes, une par une. C'était un choix : on n'automatise pas ce qu'on n'a jamais fait soi-même, et la ligne de commande oblige à nommer chaque paramètre qu'une interface graphique aurait réglé en silence.

Mais une infrastructure construite à la main souffre de deux maux qui la rendent inacceptable en entreprise. Elle n'est pas reproductible — personne ne saurait la recréer à l'identique, pas même son auteur six mois plus tard. Et elle n'est pas auditable — il n'existe nulle part un document disant ce qui *devrait* exister, donc aucun moyen de constater une dérive.

À partir d'ici, chaque ressource du laboratoire est décrite dans un fichier. Ce fichier est la vérité ; Azure n'en est que le reflet. C'est tout l'objet de Terraform.

4.2 Le carnet, et ce qu'il contient

Terraform compare ce que décrit le code avec ce qui existe réellement, puis applique la différence. Pour cette comparaison, il lui faut savoir quelles ressources sont les siennes. Il tient donc un inventaire, que l'on appelle le *state*.

Ce n'est pas un journal : le state ne raconte rien, il décrit ce qui est. Sa fonction est de séparer *à moi* de *pas à moi* — distinction dont ce laboratoire a un besoin immédiat, puisque l'abonnement héberge aussi des ressources créées pendant la préparation d'une certification. Un Terraform sans mémoire est un Terraform qui veut tout recréer.

Par défaut, ce carnet est un fichier posé à côté du code. Et c'est un problème, pour une raison qu'on ne découvre généralement qu'après l'avoir commise : Terraform y recopie les secrets que Azure lui a remis. Les clés, les mots de passe engendrés, les chaînes de connexion. En clair. Alors même qu'ils n'apparaissent nulle part dans le code écrit.

Un dépôt public et un carnet versionné font donc, ensemble, exactement le scénario d'attaque décrit au chapitre précédent. Non par négligence spectaculaire, mais par ignorance d'un détail d'implémentation.

> *Le carnet mérite le même traitement qu'une base de données de production : chiffré, à accès nominatif, journalisé, versionné. Beaucoup d'équipes le traitent comme un fichier de travail. C'est l'une des erreurs de configuration les plus répandues dans les projets Terraform, et l'une des plus silencieuses.*

4.3 L'œuf et la poule

Le carnet doit donc vivre dans Azure, dans un compte de stockage. Mais qui crée ce compte de stockage ? Pas Terraform : il lui faudrait déjà un carnet pour s'en souvenir, et le carnet n'existe pas encore.

Ce paradoxe de l'amorçage n'a pas de solution élégante. Il a des réponses assumées. Celle retenue ici est la plus simple : créer ce compte à la main, une seule fois, et écrire noir sur blanc dans la documentation qu'il échappe à la gestion par le code. Une ressource non gérée qu'on déclare vaut mieux qu'une dépendance circulaire qu'on dissimule.

En entreprise, cette ressource est généralement fournie par l'équipe plateforme, avant que le premier projet n'existe. La question ne se pose donc jamais aux développeurs — ce qui explique pourquoi tant d'ingénieurs n'ont jamais eu à y répondre.

4.4 Ce qui se joue dans un paramètre

Le compte de stockage a été verrouillé à sa création, pas après. L'ordre compte : une ressource ouverte même quelques minutes est une ressource qui a été ouverte.

Quatre verrous, dont trois vont de soi : pas d'accès anonyme, transit chiffré, versions obsolètes de TLS refusées. Le quatrième mérite qu'on s'y arrête, parce qu'il est au centre du parti pris de ce travail.

Un compte de stockage Azure expose, par défaut, deux clés d'accès. Quiconque détient l'une de ces clés *est* le compte de stockage. Il n'a pas d'identité : les journaux enregistrent un accès par clé partagée, sans sujet. Il ne peut pas être révoqué individuellement : on régénère la clé, et on casse l'accès de tout le monde. Et la clé n'expire jamais.

Désactiver ces clés change la nature de l'objet. Le carnet ne s'ouvre plus qu'avec une identité Entra ID — une personne nommée, ou l'identité fédérée du pipeline. Chaque lecture laisse un nom.

Sans ce paramètre, le carnet aurait été le seul composant du laboratoire dont les accès seraient restés anonymes. Soit précisément l'angle mort qui a motivé tout ce projet : le jour de l'incident, personne n'aurait pu dire qui avait lu la carte de l'infrastructure.

4.5 Administrer n'est pas lire

Désactiver les clés a eu une conséquence immédiate et instructive. Propriétaire de l'abonnement, j'étais incapable de créer un simple répertoire dans ce compte de stockage.

Azure sépare en effet deux plans. Le plan de contrôle gouverne l'existence et la configuration des ressources : c'est lui que *Owner* et *Contributor* administrent. Le plan de données gouverne leur contenu, et réclame des rôles distincts. Être propriétaire du coffre ne fait pas de vous le porteur de la clé des tiroirs.

Cette séparation éclaire rétrospectivement le danger des clés d'accès : elles donnaient le plan de données en contournant entièrement le contrôle d'accès basé sur les rôles. Elles ne complétaient pas RBAC, elles le court-circuitaient.

4.6 Ne pas publier la carte

Reste à dire à Terraform où se trouve son carnet. L'adresse comporte le nom du compte de stockage — et ce dépôt est public.

Ce nom n'est pas un secret : il ne donne accès à rien, la serrure tient sans lui. Mais le publier revient à distribuer la cartographie de la cible, et un nom de compte de stockage se résout publiquement sur internet. On a donc coupé la configuration en deux : le code versionné déclare la nature du backend, un fichier local non versionné en fournit les coordonnées. L'identifiant d'abonnement suit la même règle, par variable d'environnement.

Un fichier d'exemple, lui, est publié. Il montre la forme de l'adresse sans jamais sa valeur. Ce n'est pas de la décoration : c'est ce qui rend le dépôt reprenable. Un projet d'infrastructure dont on ne peut pas deviner la configuration manquante est un projet qu'on ne peut pas cloner utilement.

4.7 Une ligne qui vaut une signature

L'initialisation a produit un fichier de verrouillage, qui fige la version exacte du traducteur Azure et son empreinte cryptographique. Ce fichier-là est versionné, et c'est délibéré.

Il garantit que le pipeline du bloc 4 exécutera le même binaire que le poste de travail. Sans lui, une version publiée entre-temps — compromise ou simplement différente — s'installerait silencieusement. C'est une protection de chaîne d'approvisionnement, de la même famille que les empreintes de dépendances applicatives, et elle coûte exactement un fichier de quelques lignes.

Le laboratoire possède désormais une mémoire, rangée là où il faut, qu'on ne peut ouvrir qu'en déclinant son identité. Rien n'est encore déployé. Mais tout ce qui le sera, à partir de maintenant, sera écrit avant d'être créé.

Journal et corrections

Conserver les erreurs fait partie de la démarche. Un document qui ne montre que le résultat final n'apprend rien sur la manière d'y arriver.

Le fichier d'exclusion qui n'excluait rien. Le fichier destiné à empêcher les secrets de partir dans le dépôt public avait été nommé sans son point initial. Son contenu était juste ; l'outil ne le lisait pas. Un contrôle qui existe sans être actif est pire qu'un contrôle absent, parce qu'il procure un sentiment de sécurité que rien ne justifie. Il a été corrigé avant tout déploiement — et vérifié, ce qui n'est pas la même chose.

Un article de loi cité de travers. Le refus par défaut avait été rattaché à l'article 5 du RGPD au lieu de l'article 25. L'erreur a été corrigée dans la documentation et a produit, en creux, le développement historique du chapitre 2. Citer un texte approximativement coûte plus cher que de ne pas le citer.

Un budget au mauvais niveau. La première alerte budgétaire a été créée au niveau du compte de facturation plutôt que de l'abonnement. Elle fonctionne, mais n'est pas reproductible par le code d'infrastructure. La limite a été consignée et sa correction planifiée, plutôt que dissimulée.

Cent lignes de trace pour une coupure réseau. Une commande d'authentification a échoué en produisant une centaine de lignes de trace. La cause tenait en une ligne, tout en haut : un nom de domaine non résolu, sur une connexion par partage de réseau mobile. L'incident n'a rien coûté, mais il a imposé une discipline : une trace d'exception se lit de haut en bas, la cause première est la première exception, pas la dernière. Un ingénieur qui lit par la fin diagnostique le symptôme.

Des erreurs de frappe, et ce qu'elles démontrent. Cinq commandes sur six d'une même séance ont échoué sur un mot mal orthographié — un paramètre, un nom de clé, une valeur booléenne. Aucune n'a eu de conséquence : les outils ont refusé net. C'est précisément l'argument en faveur du code d'infrastructure. Un nom écrit une fois dans un fichier relu, versionné et formaté automatiquement ne se retape pas à chaque exécution. La classe d'erreur disparaît, elle n'est pas seulement rattrapée.

Ce document est mis à jour à chaque étape franchie. La documentation technique correspondante est tenue séparément, dans le dépôt du projet.